

系统开发工具基础

第 2 周实验报告

授课教师 周小伟

姓 名 居里

学 号 25060021025

实验日期 2026 年 8 月 31 日

授课内容 命令行操作 • Debugging • Profilling • 开发环境与工具

GitHub 仓库 https://github.com/oranlj/sdt_course

2026 年 9 月 11 日

目录

1	仓库与提交记录	2
2	课上检查题	2
2.1	第 5 题：控制一个可清理的后台任务	2
2.2	第 6 题：语义重构与本地开发反馈	4
2.3	第 7 题：用调试器定位归并排序缺陷	7
2.4	第 8 题先测量，再优化慢速词频程序	9
3	课后练习	11
3.1	命令行环境	11
3.2	Debugging & Profiling 课后练习	12
3.3	开发环境与工具课后练习	15
4	解题感悟	15
4.1	课堂检查题体会	15
4.2	课后练习体会	16
4.3	总结与改进	16

1 仓库与提交记录

GitHub 仓库: https://github.com/oranlj/sdt_course

```
commit 806dec53cc0081d09a9336fc88c03d6b38580ba
Author: remus <remus@example.com>
Date: Thu Sep 10 17:37:31 2026 +0800

    完成第二周课后练习

commit 76b4377df796ec9bc57a6e6366399f74c93e1a34
Author: remus <remus@example.com>
Date: Wed Sep 9 23:43:58 2026 +0800

    完成练习至第八题

commit 145bbff1bd9876bf4dfd9637d06869e4cd292d78
Author: remus <remus@example.com>
Date: Mon Sep 7 20:45:09 2026 +0800

    完成q08

commit 3269dd4f77abf14160bf79f2a144ddb0b6c8a856
Author: remus <remus@example.com>
Date: Mon Sep 7 16:16:47 2026 +0800

    完成q07

commit 48d06e3e0cbdf3eb2966fa36c744240d51614f
Author: remus <remus@example.com>
Date: Mon Sep 7 15:25:58 2026 +0800

    完成q06

commit 219f8ff53f7330d2ba4e7e7b8aa5b46cc5e90d09
Author: remus <remus@example.com>
Date: Mon Sep 7 11:59:37 2026 +0800

    Add Python gitignore

commit 65472a9df107057d8b4db1f1ad071496235f364d
Author: remus <remus@example.com>
Date: Sat Sep 5 23:49:17 2026 +0800

    完成q05
```

图 1: Git 提交记录截图

2 课上检查题

2.1 第 5 题：控制一个可清理的后台任务

主题：命令行环境：进程、信号与任务控制

题目要求 将脚本保存为 q05/worker.sh，再完成任务控制。

过程展示 创建并进入目录

```
1 mkdir -p q05
```

```
2 cd q05
```

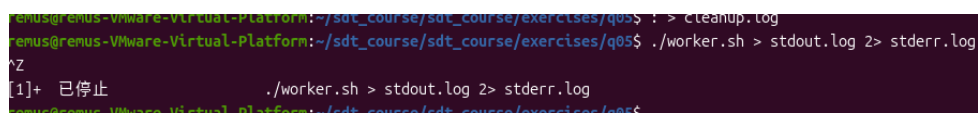
然后将内容粘贴进脚本。随后赋予权限

```
1 chmod +x worker.sh
```

在前台启动脚本并分别重定向输出

```
1 ./worker.sh > stdout.log 2> stderr.log
```

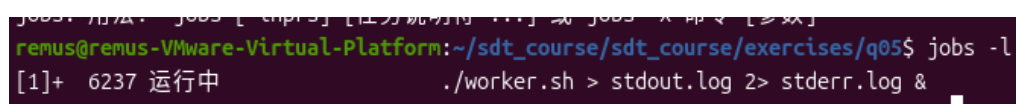
用 Ctrl-Z 挂起任务，终端显示



```
remus@remus-VMware-Virtual-Platform:~/sdt_course/sdt_course/exercises/q05$ ! > cleanup.log
remus@remus-VMware-Virtual-Platform:~/sdt_course/sdt_course/exercises/q05$ ./worker.sh > stdout.log 2> stderr.log
^Z
[1]+ 已停止 ./worker.sh > stdout.log 2> stderr.log
remus@remus-VMware-Virtual-Platform:~/sdt_course/sdt_course/exercises/q05$
```

图 2: 挂起任务

用 bg 使其在后台继续运行，然后用 Jobs 确认状态，终端显示



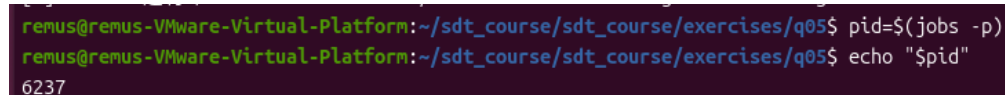
```
remus@remus-VMware-Virtual-Platform:~/sdt_course/sdt_course/exercises/q05$ jobs -l
[1]+ 6237 运行中 ./worker.sh > stdout.log 2> stderr.log &
```

图 3: 确认状态

用 jobs -p 自动取得 pid

```
1 pid=$(jobs -p %1)
```

确认取得 pid



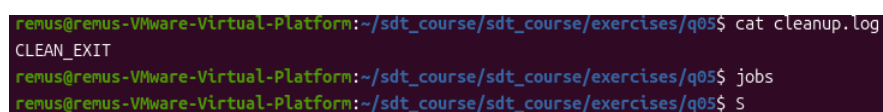
```
remus@remus-VMware-Virtual-Platform:~/sdt_course/sdt_course/exercises/q05$ pid=$(jobs -p)
remus@remus-VMware-Virtual-Platform:~/sdt_course/sdt_course/exercises/q05$ echo "$pid"
6237
```

图 4: 取得 Pid

发送 sigterm 并等待任务结束

```
1 kill -TERM "$pid"
2 wait %1
```

验证清理操作和任务状态



```
remus@remus-VMware-Virtual-Platform:~/sdt_course/sdt_course/exercises/q05$ cat cleanup.log
CLEAN_EXIT
remus@remus-VMware-Virtual-Platform:~/sdt_course/sdt_course/exercises/q05$ jobs
remus@remus-VMware-Virtual-Platform:~/sdt_course/sdt_course/exercises/q05$
```

图 5: 验证清理操作和任务状态

2.2 第 6 题：语义重构与本地开发反馈

主题：开发环境与工具

题目要求 在 q06 中按照给定代码创建三个 Python 文件，并使用编辑器语言服务器完成一次语义重构。

过程展示 F2 即可跳转到定义。然后查找引用：

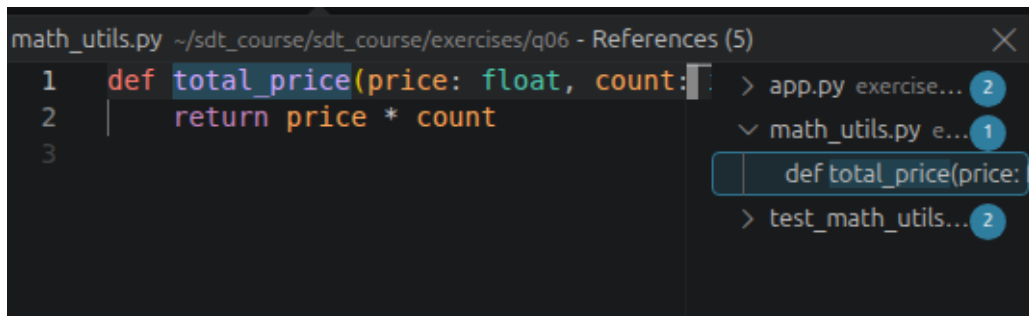


图 6: 定义、引用

使用重命名符号把 `total_price` 改为 `calculate_total`

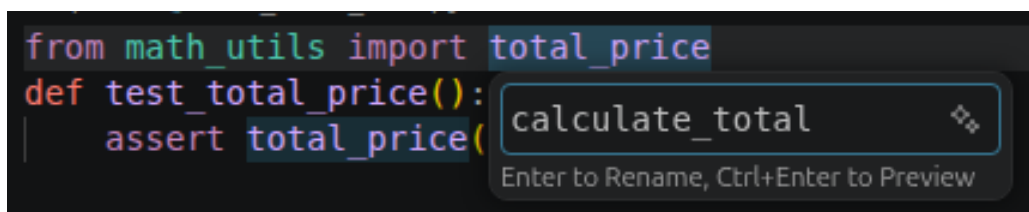
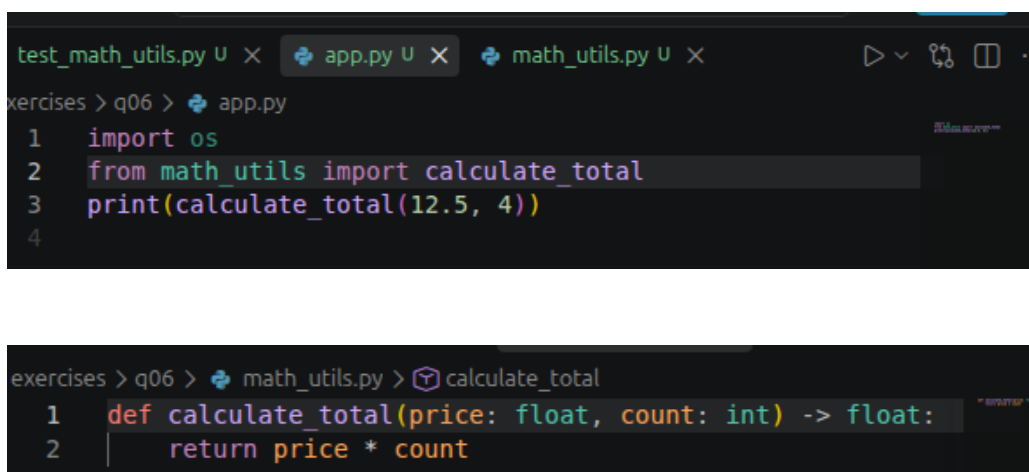


图 7: 重命名

保证定义和所有引用同步改变



```
from math_utils import calculate_total
def test_total_price():
    assert calculate_total(12.5, 4) == 50.0
```

已在 app.py 中临时加入未使用的 import os，现在用 ruff 发现并修复；最后运行 ruff 和 pytest 并保证通过。

```

I001 [*] Import block is un-sorted or un-formatted
--> app.py:1:1
|
1 | / import os
2 | | from math_utils import calculate_total
  | | ^
3 | | print(calculate_total(12.5, 4))
  | |
help: Organize imports
|
1 | import os
2 +
3 | from math_utils import calculate_total
4 +
5 | print(calculate_total(12.5, 4))
  |

F401 [*] `os` imported but unused
--> app.py:1:8
|
1 | import os
  |      ^^
2 | from math_utils import calculate_total
3 | print(calculate_total(12.5, 4))
  |
help: Remove unused import: `os`
|
- import os
1 | from math_utils import calculate_total
  |

I001 [*] Import block is un-sorted or un-formatted
--> test_math_utils.py:1:1
|
1 | from math_utils import calculate_total
  | ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
2 | def test_total_price():
3 |     assert calculate_total(12.5, 4) == 50.0
  |
help: Organize imports
|
1 | from math_utils import calculate_total
2 +
3 +
4 | def test_total_price():
  |

Found 3 errors.
[*] 3 fixable with the `--fix` option.

```

图 8: ruff 发现问题 (注: 之前复制代码时候缩进出现问题, ruff 在这里一并发现, 后续参与修复)

```
(.venv) remus@remus-VMware-Virtual-Platform:~/sdt_course/sdt_course/exercises/q06$ ruff check . --fix
Found 3 errors (3 fixed, 0 remaining).
```

图 9: ruff 修复

```
(.venv) remus@remus-VMware-Virtual-Platform:~/sdt_course/sdt_course/exercises/q05$ pytest
platform linux -- Python 3.14.4, pytest-9.1.1, pluggy-1.6.0
rootdir: /home/remus/sdt_course/sdt_course/exercises/q05
collected 3 items

test_math_util.py ..... [passed] [ 100%]
```

图 10: pytest 通过

2.3 第 7 题：用调试器定位归并排序缺陷

主题: Debugging

题目要求 将下列存在缺陷的归并排序代码保存为 q07/merge_sort.py，再使用调试器定位并修复。。

操作步骤 初步运行发现问题，排序结果错误

```
(.venv) remus@remus-VMware-Virtual-Platform:~/sdt_course/sdt_course/exercises/q07$ python merge_sort.py
[1, 5, 4, 3, 4, 5, 6, 9]
```

图 11: 运行结果

添加断点

```
1 def merge(left, right):
2     result = []
3     i = j = 0
4
5     breakpoint()
6
7     while i < len(left) and j < len(right):
8         if left[i] <= right[j]:
9             result.append(left[i]); i += 1
10        else:
11            result.append(right[i]); j += 1 # 此处存在缺陷
12    return result + left[i:] + right[j:]
```

使用 pdb 定位错误。在 merge 函数中设置 breakpoint() 后，使用 pdb 单步执行并观察 i、j、left[i]、right[j]。当运行到 left=[1,3]、right=[1,4] 时，发现此时 i=1、j=0，应比较并取出 right[j]=1，但程序却执行了 right[i]=4，且随后增加的是 j。由此可判断右侧列表的索引应使用 j 而不是 i。

```

n. 未找到命令
(.venv) remus@remus-VMware-Virtual-Platform:~/sdt_course/sdt_course/exercises/q07$ python merge_sort.py
> /home/remus/sdt_course/sdt_course/exercises/q07/merge_sort.py(5)merge()
-> breakpoint()
(Pdb) p i, j, left[i], right[j]
(0, 0, 3, 1)
(Pdb) c
> /home/remus/sdt_course/sdt_course/exercises/q07/merge_sort.py(5)merge()
-> breakpoint()
(Pdb) p i, j, left[i], right[j]
(0, 0, 4, 1)
(Pdb) c
> /home/remus/sdt_course/sdt_course/exercises/q07/merge_sort.py(5)merge()
-> breakpoint()
(Pdb) p i, j, left[i], right[j]
(0, 0, 1, 1)
(Pdb) n
> /home/remus/sdt_course/sdt_course/exercises/q07/merge_sort.py(7)merge()
-> while i < len(left) and j < len(right):
(Pdb) n
> /home/remus/sdt_course/sdt_course/exercises/q07/merge_sort.py(8)merge()
-> if left[i] <= right[j]:
(Pdb) p left[i] <= right[j]
True
(Pdb) n
> /home/remus/sdt_course/sdt_course/exercises/q07/merge_sort.py(9)merge()
-> result.append(left[i]); i += 1
(Pdb) p result
[]
(Pdb) p i, j
(0, 0)
(Pdb) n
> /home/remus/sdt_course/sdt_course/exercises/q07/merge_sort.py(7)merge()
-> while i < len(left) and j < len(right):
(Pdb) p result
[1]
(Pdb) p i, j
(1, 0)
(Pdb) p left[i], right[i]
(3, 4)
(Pdb) n
> /home/remus/sdt_course/sdt_course/exercises/q07/merge_sort.py(8)merge()
-> if left[i] <= right[j]:
(Pdb) p left[i] <= right[j]
False
(Pdb) n
> /home/remus/sdt_course/sdt_course/exercises/q07/merge_sort.py(11)merge()
-> result.append(right[i]); j += 1 # 此处存在缺陷
(Pdb) p i, j
(1, 0)
(Pdb) p right[i], right[j]
(4, 1)

```

图 12: PDB 定位错误

因此完成修复：result.append(right[i]) 修改为 result.append(right[j])。

随后进行 pytest 测试

```

1 from merge_sort import merge_sort
2
3 def test_given_input():
4     assert merge_sort([3, 1, 4, 1, 5, 9, 2, 6]) == [1, 1, 2, 3, 4, 5, 6, 9]
5

```

```

6 def test_duplicates():
7     assert merge_sort([5, 2, 5, 1, 2, 1]) == [1, 1, 2, 2, 5, 5]

```

Listing 1: test_merge_sort.py

运行 pytest，全部通过

```

(.venv) renus@renus-Virtual-Platform:~/sdt_course/sdt_course/exercises/q07$ pytest
platform linux -- Python 3.14.4, pytest-9.1.1, pluggy-1.6.0
rootdir: /home/renus/sdt_course/sdt_course/exercises/q07
collected 2 items

test_merge_sort.py ..                                     [100%]

===== 2 passed in 0.00s =====

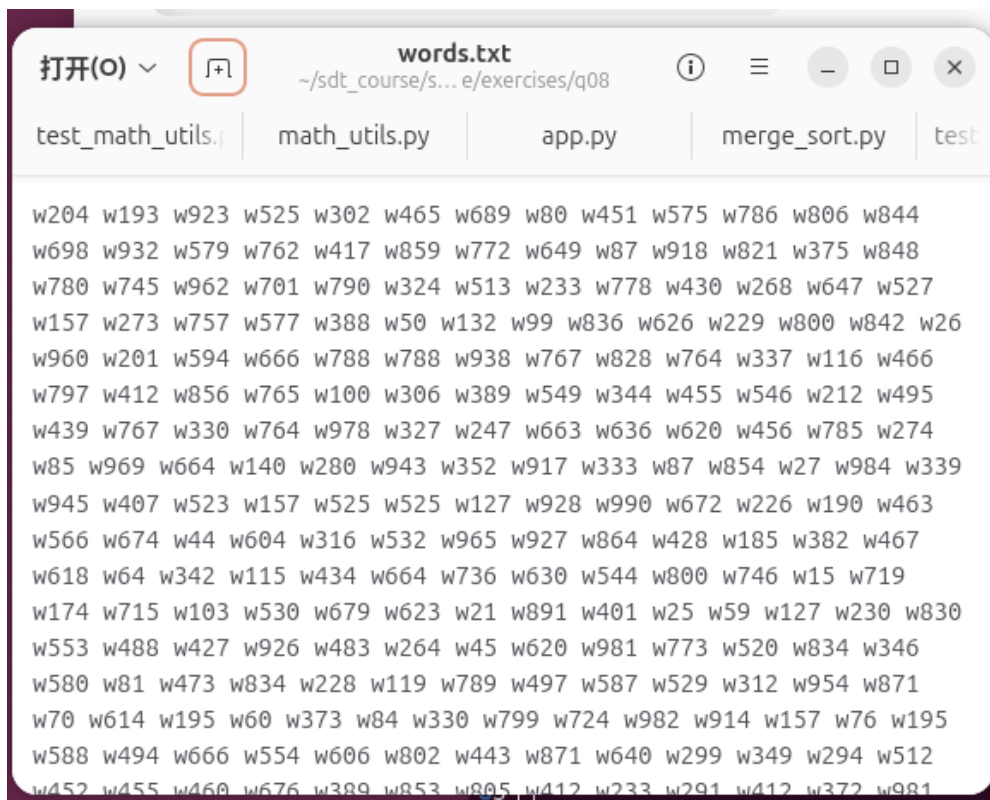
```

2.4 第 8 题先测量，再优化慢速词频程序

主题: Profiling

题目要求 在 q08 中创建可重复生成的词表和故意低效的去重程序，再使用性能分析工具优化。

实现过程 运行 generate_words.py，得到 words.txt



words.txt

test_math_utils.py | math_utils.py | app.py | merge_sort.py | test

w204 w193 w923 w525 w302 w465 w689 w80 w451 w575 w786 w806 w844
w698 w932 w579 w762 w417 w859 w772 w649 w87 w918 w821 w375 w848
w780 w745 w962 w701 w790 w324 w513 w233 w778 w430 w268 w647 w527
w157 w273 w757 w577 w388 w50 w132 w99 w836 w626 w229 w800 w842 w26
w960 w201 w594 w666 w788 w788 w938 w767 w828 w764 w337 w116 w466
w797 w412 w856 w765 w100 w306 w389 w549 w344 w455 w546 w212 w495
w439 w767 w330 w764 w978 w327 w247 w663 w636 w620 w456 w785 w274
w85 w969 w664 w140 w280 w943 w352 w917 w333 w87 w854 w27 w984 w339
w945 w407 w523 w157 w525 w525 w127 w928 w990 w672 w226 w190 w463
w566 w674 w44 w604 w316 w532 w965 w927 w864 w428 w185 w382 w467
w618 w64 w342 w115 w434 w664 w736 w630 w544 w800 w746 w15 w719
w174 w715 w103 w530 w679 w623 w21 w891 w401 w25 w59 w127 w230 w830
w553 w488 w427 w926 w483 w264 w45 w620 w981 w773 w520 w834 w346
w580 w81 w473 w834 w228 w119 w789 w497 w587 w529 w312 w954 w871
w70 w614 w195 w60 w373 w84 w330 w799 w724 w982 w914 w157 w76 w195
w588 w494 w666 w554 w606 w802 w443 w871 w640 w299 w349 w294 w512
w452 w455 w460 w676 w389 w853 w805 w412 w233 w291 w412 w372 w981

图 13: words.txt

time 运行两次，记录耗时分别为 0.087s, 0.084s

```
(.venv) remus@remus-Virtual-Platform:~/sdt_course/sdt_course/exercises/q0$ time python wordfreq.py
count= 1000

real    0m0.087s
user    0m0.080s
sys     0m0.006s
(.venv) remus@remus-Virtual-Platform:~/sdt_course/sdt_course/exercises/q0$ time python wordfreq.py
count= 1000

real    0m0.084s
user    0m0.079s
```

图 14: 耗时

然后 cProfile -s cumulative 运行一次，找到“慢在哪里”

```
(.venv) remus@remus-Virtual-Platform:~/sdt_course/sdt_course/exercises/q0$ python -m cProfile -s cumulative wordfreq.py
count= 1000
1012 function calls in 0.072 seconds

Ordered by: cumulative time

ncalls  tottime  percall  cumtime  percall filename:lineno(function)
1      0.000   0.000   0.072    0.072 {built-in method builtins.exec}
1      0.071   0.071   0.072    0.072 wordfreq.py:1(<module>)
1      0.001   0.001   0.001   0.001 {method 'split' of 'str' objects}
1      0.000   0.000   0.000   0.000 {method 'read' of '_io.TextIOWrapper' objects}
1      0.000   0.000   0.000   0.000 <frozen codecs>:322(decode)
1000    0.000   0.000   0.000   0.000 {method 'append' of 'list' objects}
1      0.000   0.000   0.000   0.000 {built-in method _codecs.utf_8_decode}
1      0.000   0.000   0.000   0.000 {built-in method builtins.print}
1      0.000   0.000   0.000   0.000 {method 'disable' of '_lsprof.Profiler' objects}
1      0.000   0.000   0.000   0.000 {built-in method _io.open}
1      0.000   0.000   0.000   0.000 {built-in method builtins.len}
1      0.000   0.000   0.000   0.000 <frozen codecs>:312(__init__)
1      0.000   0.000   0.000   0.000 <frozen codecs>:263(__init__)
```

图 15: words.txt

wordfreq.py 模块自身耗时 0.071 s，而文件读取、字符串分割和 1000 次 list.append() 的耗时均很小，说明主要耗时集中在循环内部。结合代码分析可知，word not in unique 对列表需要进行线性查找，是主要性能瓶颈。因此将 unique 从 list 改为 set。

```
1 words = open("words.txt", encoding="utf-8").read().split()
2 unique = set()
3 for word in words:
4     if word not in unique:
5         unique.add(word)
6 print("count=", len(unique))
```

Listing 2: 修改后的 wordfreq.py

重新 time 跑两次，发现运行时间大幅缩短从平均 0.0855s 缩短到 0.0135s。

```
(.venv) remus@remus-Virtual-Platform:~/sdt_course/sdt_course/exercises/q0$ time python wordfreq.py
count= 1000

real    0m0.013s
user    0m0.010s
sys     0m0.003s
(.venv) remus@remus-Virtual-Platform:~/sdt_course/sdt_course/exercises/q0$ time python wordfreq.py
count= 1000

real    0m0.014s
user    0m0.008s
sys     0m0.006s
```

图 16: report.pdf 渲染效果

3 课后练习

3.1 命令行环境

1. `-` 参数试着运行 `touch - -myfile`，然后在不使用 `-` 的情况下把它删掉

```
1 touch -- -myfile
2 rm -- -myfile
```

2. `ls` 用法写出一个 `ls` 命令，让它按下面的方式列出文件：包含所有文件，包括隐藏文件文件大小以易读格式显示（例如 454M，而不是 454279954）按最近修改时间排序输出带颜色

```
1 ls -laht --color=auto
```

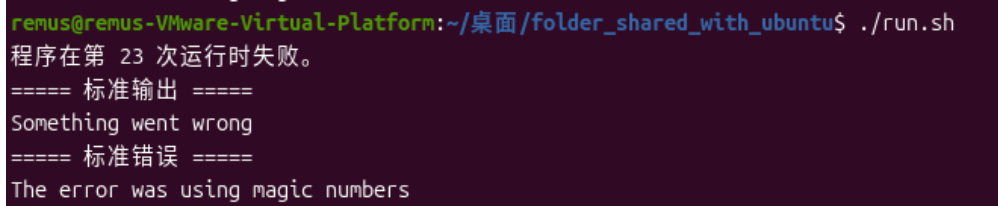
3. 环境变量写两个 `bash` 函数 `marco` 和 `polo`，行为如下：每次执行 `marco` 时，都要以某种方式保存当前工作目录；之后无论你切到哪个目录，只要执行 `polo`，它都应该把你 `cd` 回执行 `marco` 时所在的目录。为了方便调试，把代码写进 `marco.sh`，然后通过执行 `source marco.sh` 把这些定义重新加载到当前 `shell`。

```
1 mkdir -p glob-test && cd glob-test
2 marco() {
3     MARCO_DIR="$PWD"
4 }
5
6 polo() {
7     cd "$MARCO_DIR"
8 }
```

4. 返回码写一个 `bash` 脚本，不断运行某脚本（`example.sh`）直到它失败为止，并把标准输出和标准错误分别保存到文件里，最后把结果打印出来并报告它运行了多少次才失败。

```
1 #!/usr/bin/env bash
2
3 count=0
4
5 while true; do
6     ((count++))
7
8     ./example.sh > stdout.txt 2> stderr.txt
9
10    if [[ $? -ne 0 ]]; then
```

```
11 break
12 fi
13 done
14
15 echo "程序在第 $count 次运行时失败。"
16 echo "==== 标准输出 ====="
17 cat stdout.txt
18 echo "==== 标准错误 ====="
19 cat stderr.txt
```



```
remus@remus-VMware-Virtual-Platform:~/桌面/folder_shared_with_ubuntu$ ./run.sh
程序在第 23 次运行时失败。
==== 标准输出 =====
Something went wrong
==== 标准错误 =====
The error was using magic numbers
```

图 17: 第四题运行结果图

3.2 Debugging & Profiling 课后练习

5. 使用 AddressSanitizer 调试释放后使用错误将题目给出的程序保存为 `uaf.c`。普通编译运行时，程序打印不符合预期。随后加入 AddressSanitizer 重新编译：

```
1 gcc uaf.c -o uaf && ./uaf
2 gcc -fsanitize=address -g uaf.c -o uaf_asan
3 ./uaf_asan
```

AddressSanitizer 报告 `heap-use-after-free`，指出第 9 行释放了内存，第 10 行仍通过原指针写入。修复时把字符修改移动到 `free` 之前，并在释放后把指针设为 `NULL`。修复版本符合预期

```

remus@remus-Virtual-Platform:~/桌面/sdt_course/after_class/week2/ex05$ ./uaf_asan
Hello, world!
=====
==999==ERROR: AddressSanitizer: heap-use-after-free on address 0x7a224fbc0040 at pc 0x5668a8b612ca bp 0x7ffc0e3aa6b0 sp 0x7ffc0e3aa6a0
WRITE of size 1 at 0x7a224fbc0040 thread T0
#0 0x5668a8b612c9 in main /home/remus/桌面/sdt_course/after_class/week2/ex05/uaf.c:10
#1 0x7df25082a600 in __libc_start_call_main ../sysdeps/nptl/libc_start_call_main.h:59
#2 0x7df25082a717 in __libc_start_main_impl ../csu/libc-start.c:360
#3 0x5668a8b61184 in _start (/home/remus/sdt_course/sdt_course/after_class/week2/ex05/uaf_asan+0x1184) (BuildId: 6d31cc2d56e4a5a9af11b8aa08d159b5f32d1bff)

0x7a224fbc0040 is located 0 bytes inside of 32-byte region [0x7a224fbc0040,0x7a224fbc0060)
freed by thread T0 here:
#0 0x7df250d2a3ff in free ../../src/libsanitizer/asan/asan_malloc_linux.cpp:51
#1 0x5668a8b61295 in main /home/remus/桌面/sdt_course/after_class/week2/ex05/uaf.c:9
#2 0x7df25082a600 in __libc_start_call_main ../sysdeps/nptl/libc_start_call_main.h:59
#3 0x7df25082a717 in __libc_start_main_impl ../csu/libc-start.c:360
#4 0x5668a8b61184 in _start (/home/remus/sdt_course/sdt_course/after_class/week2/ex05/uaf_asan+0x1184) (BuildId: 6d31cc2d56e4a5a9af11b8aa08d159b5f32d1bff)

previously allocated by thread T0 here:
#0 0x7df250d2b60f in malloc ../../src/libsanitizer/asan/asan_malloc_linux.cpp:67
#1 0x5668a8b6125e in main /home/remus/桌面/sdt_course/after_class/week2/ex05/uaf.c:6
#2 0x7df25082a600 in __libc_start_call_main ../sysdeps/nptl/libc_start_call_main.h:59
#3 0x7df25082a717 in __libc_start_main_impl ../csu/libc-start.c:360
#4 0x5668a8b61184 in _start (/home/remus/sdt_course/sdt_course/after_class/week2/ex05/uaf_asan+0x1184) (BuildId: 6d31cc2d56e4a5a9af11b8aa08d159b5f32d1bff)

SUMMARY: AddressSanitizer: heap-use-after-free /home/remus/桌面/sdt_course/after_class/week2/ex05/uaf.c:10 in main
Shadow bytes around the buggy address:
 0x7a224fbd0000: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
 0x7a224fbd0008: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
 0x7a224fbd0010: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
 0x7a224fbd0018: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
 0x7a224fbd0020: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
 0x7a224fbd0028: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
=>0x7a224fbc0040: fa fa 00 00 00 fa fa fa[fd]fd fd fd fa fa fa
0x7a224fbc0048: fa fa fa fa fa fa fa fa fa fa fa fa fa fa fa fa
0x7a224fbc0050: fa fa fa fa fa fa fa fa fa fa fa fa fa fa fa fa
0x7a224fbc0058: fa fa fa fa fa fa fa fa fa fa fa fa fa fa fa fa
0x7a224fbc0060: fa fa fa fa fa fa fa fa fa fa fa fa fa fa fa fa
0x7a224fbc0068: fa fa fa fa fa fa fa fa fa fa fa fa fa fa fa fa
Shadow byte legend (one shadow byte represents 8 application bytes):
Addressable: 00
Partially addressable: 01 02 03 04 05 06 07
Heap left redzone: fa
Freed heap region: fd
Stack left redzone: f1
Stack mid redzone: f2
Stack right redzone: f3
Stack after return: f5
Stack use after scope: f8
Global redzone: f9
Global init order: f6
Poisoned by user: f7
Container overflow: fc
Array cookie: ac
Intra object redzone: bb
Asan internal: fe
Left alloc redzone: ca

```

图 18: AddressSanitizer 定位错误并验证修复

6. 使用 strace 分析系统调用

先跟踪 `ls -l` 的文件相关系统调用，再编写读取 `sample.txt` 的小程序并观察其打开、读取和关闭文件的过程：

```

1 strace -e trace=openat,newfstatat,getdents64,close ls -l
2 strace -e trace=openat,read,close ./open_files
3 strace -c ./open_files

```

结果表明，`ls` 使用 `openat` 打开目录和共享库，使用 `getdents64` 读取目录项；自编写程序通过 `openat` 打开 `sample.txt`，通过 `read` 读取 25 字节，最后用 `close` 释放文件描述符。`strace -c` 还能汇总各系统调用的次数和耗时。

7. 定位并释放被占用的网络端口

在 4444 端口启动 Python HTTP 服务，使用 `ss` 找出监听进程，再终止该进程并检查端口状态：

```

1 python3 -m http.server 4444 &
2 ss -tlnp | grep ':4444'

```

```

3 kill <PID>
4 ss -tln | grep ':4444'

```

Listing 3: 关键指令

`ss` 显示监听程序为 `python3`，进程号为 5968。访问本地服务得到 HTTP 200；执行 `kill` 后再次查询，确认 4444 端口已经释放。

```

remus@remus-VMware-Virtual-Platform:~/桌面/sdt_course/after_class/wee
k2/ex07$ python3 -m http.server 4444 > server.log 2>&1 &
[1] 6209
remus@remus-VMware-Virtual-Platform:~/桌面/sdt_course/after_class/wee
k2/ex07$ ss -tlnp | grep ':4444'
LISTEN 0      5            0.0.0.0:4444      0.0.0.0:*      users:(("python3",pid=6209,fd=3))
remus@remus-VMware-Virtual-Platform:~/桌面/sdt_course/after_class/wee
k2/ex07$ kill 6209
[1]+  已终止                  python3 -m http.server 4444 > server.log 2>&1
remus@remus-VMware-Virtual-Platform:~/桌面/sdt_course/after_class/wee
k2/ex07$ ss -tlnp | grep ':4444'
remus@remus-VMware-Virtual-Platform:~/桌面/sdt_course/after_class/week2/ex07$

```

图 19: 使用 `ss` 查找端口占用进程并释放端口

8. 使用 `hyperfine` 比较文本搜索实现

在 `ex08` 中生成 40 个日志文件，总大小约 2.7 MB，比较递归 `grep` 与 Python 脚本查找 `ERROR` 行的速度：

```

1 hyperfine --warmup 2 --runs 8 \
2   'grep -R ERROR data > /dev/null' \
3   'python3 search_python.py > /dev/null'

```

八次测试中，`grep` 平均耗时约 1.4 ms，Python 实现平均耗时约 25.4 ms；本次数据和环境下，`grep` 约快 18.45 倍。该结果说明基准测试应固定输入、预热程序并进行多次测量，同时应注意耗时过短时 `shell` 启动开销会影响精度。

```

remus@remus-VMware-Virtual-Platform:~/桌面/sdt_course/after_class/week2/ex07$ hyperfine --warmup 2 --runs 8 \
'grep -R ERROR data > /dev/null' \
'python3 search_python.py > /dev/null'
Benchmark 1: grep -R ERROR data > /dev/null
Time (mean ± stdev): 1.4 ms ± 0.2 ms [user: 1.4 ms, system: 0.0 ms]
Range (min - max): 1.3 ms - 1.6 ms 8 runs
Warning: Command took less than 5 ms to complete. Note that the results might be inaccurate because hyperfine can not calibrate the shell startup time much more precise than this limit. You can try to use the '-x /--shelltime' option to disable the shell completely.
Benchmark 2: python3 search_python.py > /dev/null
Time (mean ± stdev): 25.4 ms ± 2.1 ms [user: 25.4 ms, system: 0.0 ms]
Range (min - max): 23.7 ms - 27.1 ms 8 runs
Summary
'grep -R ERROR data > /dev/null' ran
25.4 times faster than 'python3 search_python.py > /dev/null'

```

图 20: `hyperfine` 比较 `grep` 与 Python 搜索性能

9. 使用 `perf stat` 获取程序性能数据

使用 `perf stat` 测量递归读取 `/usr` 目录时的性能：

```

1 perf stat ls -R /usr > /dev/null

```

程序实际运行 7.343 秒,其中用户态耗时 1.711 秒,内核态耗时 1.671 秒。`cpu-cycles` 显示 `<not supported>`, 可能是 windows 系统启用 hyper-v 的缘故。

```
remus@remus-VMware-Virtual-Platform:~/桌面/sdt_course/after_class$ perf stat ls -R /usr > /dev/null

Performance counter stats for 'ls -R /usr':

<not supported>      cpu_core/cpu-cycles/u      #      nan GHz    cycles_frequency
<not supported>      cpu_atom/cpu-cycles/u      #      nan GHz    cycles_frequency

 7.343014769 seconds time elapsed

 1.711235000 seconds user
 1.671467000 seconds sys
```

图 21: 使用 perf stat 获取程序性能数据

3.3 开发环境与工具课后练习

10. 完成一道 VimGolf 挑战: Filling braces

题目要求 初始文件包含三行: `.ocean {}`、`.land {}` 和 `.sky {}`。要求在 Vim 中分别向三对花括号内填入 `blue`、`green` 和 `lightblue`, 最终得到:

```
1 .ocean {blue}
2 .land {green}
3 .sky {lightblue}
```

使用的按键

```
1 $iblue<Esc>jigreen<Esc>jilightblue<Esc>
```

4 解题感悟

4.1 课堂检查题体会

第 5 题让我体会了前台任务、后台任务和信号之间的关系。同时了解了 `Ctrl+C`, `Ctrl+Z`、`bg`、`jobs`、`kill` 和 `wait` 各自的作用, 也认识到结束后台任务后还应检查和清理。

第 6 题让我体会到语言服务器的妙用, 感受了在编辑器中的跳转、查找引用和重命名等便捷实用。我初步学习使用 Ruff 和 pytest 等好用的开发工具。

第 7 题是一次比较完整的调试过程。面对错误的排序结果, 在 `merge` 函数中设置断点, 逐步查看 `i`、`j` 和列表元素的变化, 最后定位到右侧列表索引写错。这个过程让我理解了调试器的价值: 它可以把程序运行时的状态直接展示出来, 使错误原因更有依据。

第 8 题让我初步理解了测量与优化。通过 `time` 和 `cProfile`, 我发现程序的主要耗时位于列表检查。把列表改为集合后再次测量, 运行时间明显下降。这说明选择合适的数据结构会直接影响性能, 测量工具非常实用。

4.2 课后练习体会

命令行环境练习加深了我对参数、环境变量、退出状态和输入输出重定向的理解。例如，`--` 可以避免以短横线开头的文件名被误认为选项；Shell 函数可以保存并恢复目录；退出码能够成为脚本判断成功或失败的依据。这些练习加深了我对 shell 的认识。

Debugging 与 Profiling 练习让我接触了几类用途不同的工具。AddressSanitizer 能直接报告释放后使用，并同时给出出错、释放和分配位置；`strace` 可以观察程序与操作系统之间的系统调用；`ss` 能根据端口找到监听进程；`hyperfine` 可以通过预热和多次运行比较不同实现的速度。

VimGolf 挑战让我感受到 Vim 的编辑思路与普通文本编辑器有所不同，它使我开始尝试不再依赖鼠标，而是把文本编辑当作“移动”和“执行修改”。

4.3 总结与改进

本次实验最大的收获，是初步建立了使用工具定位问题的过程：先运行程序复现现象，再选择合适的工具收集信息，根据输出定位原因，修改后重新运行或测试。相比直接修改代码，这种方法步骤更多，但结论更可靠，也便于说明自己为什么这样修改。

我目前对调试和性能分析流程的熟悉度不够，之后我会继续练习阅读工具说明，并在实践中及时记录关键命令、现象和结论。